COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithm	**SECTION:**	CPE12S4
DATE PERFORMED:	October 16-22, 2026	**DATE SUBMITTED:**	October 22, 2026
NAME:	Cardenas, Lea Marie De Vera, Sophia	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13 PARALLEL ALGORITHMS AND MULTITHREADING

1. PROCEDURE OUTPUT

What procedures did you perform?
Create C++ code that uses modern programming techniques to implement parallel algorithms, multithreading and concurrency

ILO A & ILO B:
Part 1: Simple One-Threaded Example

SCREENSHOTS

```cpp
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

```
Output  Debug Console  Terminal  Ports
T...  .\enter program name, for example C:\Users\carde\OneDrive\Documer
PS C:\Users\carde\OneDrive\Documents\Cardenas_13.1> .\ILO
A.exe
Printing integer: 10
Printing string: T.I.P.
PS C:\Users\carde\OneDrive\Documents\Cardenas_13.1>
```

CPE010_HOA13_De Vera_sc1.cpp > main()

```cpp
#include <iostream>
#include <thread>
#include <string>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P");
    t1.join();
    return 0;
}
```

```
Printing integer: 10
Printing string: T.I.P
PS C:\DSA\HOA 13>
```

OBSERVATION ANALYSIS

- *This code demonstrates one thread, the use of single thread, that execute the print function of " 10 and T.I.P" and the join function makes sure the thread finishes before the program ends like in my screenshots.*
- *This ILO A and B focuses on basic threading where it shows how separate printing and executing of threads works. The print() makes sure that it prints both values in the program, the 10 and T.I.P. . The t1 is responsible for creating a thread which will be called once the print() is called. The rogram creates only one additional thread.*

Part 2: Multi-Threaded Example

SCREENSHOTS

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <string>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;
    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }
    for (auto &th : threads) {
        th.join();
    }
    return 0;
}
```

First Run:

```
0 : T.I.P.1 : Competent3 : Engineers

2 : Computer

PS C:\DSA\HOA 13>
```

Second Run:

```
1 : Competent3 : Engineers
2 : Computer

0 : T.I.P.
PS C:\DSA\HOA 13>
```

Third Run:

```
1 : Competent0 : T.I.P.
3 : Engineers
2 : Computer
```

ILOB.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <thread>
#include <vector>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }
    return 0;
}
```

1st run:

```
PS C:\Users\carde\OneDrive\Documents\Cardenas_13.1>
PS C:\Users\carde\OneDrive\Documents\Cardenas_13.1> .\ILOB.exe
2 : Computer
1 : Competent
3 : Engineers
0 : T.I.P.
PS C:\Users\carde\OneDrive\Documents\Cardenas_13.1>
```

2nd run:

```
PS C:\Users\carde\OneDrive\Documents\Cardenas_13.1> .\ILOB.exe
0 : T.I.P.2 : Computer3 : Engineers


1 : Competent
PS C:\Users\carde\OneDrive\Documents\Cardenas_13.1>
PS C:\Users\carde\OneDrive\Documents\Cardenas_13.1>
```

3rd run:

```
PS C:\Users\carde\OneDrive\Documents\Cardenas_13.1> .\ILOB.exe
0 : T.I.P.
3 : Engineers
2 : Computer
1 : Competent
PS C:\Users\carde\OneDrive\Documents\Cardenas 13.1>
```

OBSERVATION ANALYSIS	- ***This code demonstrates multithreading where it prints multiple or different strings and runs simultaneously or concurrently. Initial observation is that the strings do not always appear in their original order, and their output may vary each time the program runs. So I think the output displays like this***

	because the operating system of my computer somehow "doesn't know who to call first" and can't guarantee the printing of strings in order. - *The program basically creates four threads. Each string gets a number, and lets them print their messages at the same time. The functions used in this program are:* ● *push_back() → adds a thread to the vector* ● *join() → where it waits for the thread to finish* ● *s.size() → returns the number of strings in the vector*

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

COMPUTER ENGINEERING

QUEZON CITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

1. Write a definition of multithreading and its advantages/disadvantages.
   - Multithreading is a programming technique in which one process creates and runs several threads of execution. The threads share the same memory (heap and global variables) but each has its own stack and instruction pointer, so different parts of a program can make progress at the same time on multiple cores, or take turns on a single core. In C++ (since C++11) it is done with std::thread, together with tools such as std::mutex, std::condition_variable, std::atomic and std::async.

Advantages

- **Faster CPU-bound work.** Splitting work across cores shortens run time. The merge sort in Part C ran about 1.8 times faster on two cores.
- **Responsiveness.** A slow task (file, network or heavy computation) can run on a background thread while the main thread keeps responding.
- **Better throughput for waiting tasks.** While one thread waits for I/O, another can use the CPU.
- **Cheap data sharing.** Threads share memory, so exchanging data between them is cheaper than between separate processes.

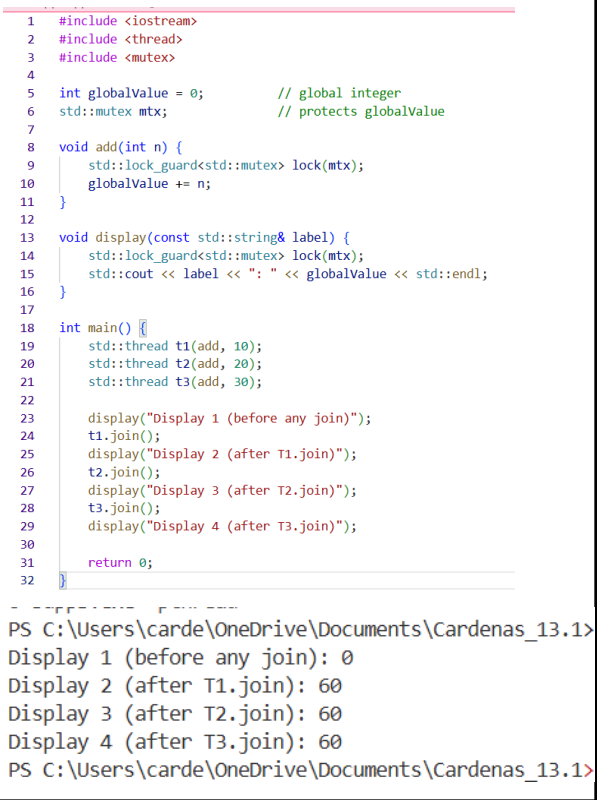
Disadvantages

- **Data races.** Unsynchronized access to shared data gives wrong results or undefined behavior (see Part B, Variant 3).
- **Deadlocks and added complexity.** Mutexes and condition variables fix races but can cause deadlock if used carelessly.
- **Overhead.** Creating threads and switching between them costs time and memory, which makes multithreading wasteful for short, simple tasks.
- **Non-deterministic behavior.** Execution order changes from run to run (see Part 2), which makes bugs hard to reproduce and test.

2. Rationalize the use of multithreading by providing at least 3 use-cases.
   - Since ***multithreading*** lets the program print values and strings concurrently—albeit with its possibility of printing the threads and strings in a jumbled order—its benefits may vary on how the tasks are distributed among threads. Through the use of threading, it can help us handle I/O resources in a single process (GeeksforGeeks, 2026). For example, here's how multithreading works in our daily tasks that we may not know,
     1. **Database and Financial Transactions** as it helps these systems handle requests concurrently, improving efficiency and responsiveness. Although the use of multithreading is not as visually obvious as printing multiple strings like in our program, it can allow tasks such as checking account balances, processing payments, and recording transactions to be handled concurrently.
     2. **Designing GUIs** wherein we typically run heavy background applications of compilers. Just like in OOP2, the application doesn't seem to freeze despite having compilation in progress because background tasks can run separately from the graphical user interface (GUI) thread.
     3. **Online gaming in PCs and Mobile Phones** where the OS of the device together with the sounds, game resources, cache files run at the same time to make out gaming experience possible.

3. Differentiate between parallelism and concurrency.
   - The difference between ***parallelism*** and ***concurrency*** differs from how they were executed and how they treat multiple tasks at a given time. Parallelism involves executing multiple tasks at the exact same time, usually using multiple CPU cores. On the other hand, concurrency involves managing multiple tasks that can make progress within overlapping periods, even if they are not executing at the exact same time. An analogy we can think of:
     - ➔ **Parallelogism are like factory stations/sections where there are programs and tasks to do different jobs at the same time. These work to allow the highest productivity the company can have.**
     - ➔ **Concurrency is like a worker doing all jobs at once and performs without any help. They switch tasks depending on what they think is highly prioritized at the time.**

Part B: Create C++ Code and show a solution that satisfies the given requirements below

Screenshots	
Analysis	It demonstrates the use of **multithreading and mutexes in C++**. Three threads (t1, t2, and t3) add 10, 20, and 30, respectively, to the global variable globalValue. The mutex protects the shared variable from being accessed or modified by multiple threads at the same time. The first display shows **0** because none of the threads has necessarily finished adding its value when the display function is called. After

```cpp
#include <iostream>
#include <thread>
#include <mutex>

int globalValue = 0;          // global integer
std::mutex mtx;               // protects globalValue

void add(int n) {
    std::lock_guard<std::mutex> lock(mtx);
    globalValue += n;
}

void display(const std::string& label) {
    std::lock_guard<std::mutex> lock(mtx);
    std::cout << label << ": " << globalValue << std::endl;
}

int main() {
    std::thread t1(add, 10);
    std::thread t2(add, 20);
    std::thread t3(add, 30);

    display("Display 1 (before any join)");
    t1.join();
    display("Display 2 (after T1.join)");
    t2.join();
    display("Display 3 (after T2.join)");
    t3.join();
    display("Display 4 (after T3.join)");

    return 0;
}
```

```
PS C:\Users\carde\OneDrive\Documents\Cardenas_13.1>
Display 1 (before any join): 0
Display 2 (after T1.join): 60
Display 3 (after T2.join): 60
Display 4 (after T3.join): 60
PS C:\Users\carde\OneDrive\Documents\Cardenas_13.1>
```

	t1.join(), the value becomes **60** because all three threads have already completed their tasks. The following displays remain at 60 because the threads have finished adding their values. This shows that join() waits for a specific thread to finish, while the mutex helps ensure safe access to shared data.

Screenshots	CPE010_HOA13_De Vera_sc3.cpp > main()
	1 #include <iostream>
	2 #include <thread>
	3 #include <mutex>
	4
	5 int total = 0;
	6 std::mutex mtx;
	7
	8 void add(int value) {
	9 std::lock_guard<std::mutex> lock(mtx);
	10 total = total + value;
	11 }
	12
	13 void display() {
	14 std::lock_guard<std::mutex> lock(mtx);
	15 std::cout << total << std::endl;
	16 }
	17
	18 int main() {
	19 std::cout << "Original Value: ";
	20 display();
	22 std::thread T1(add, 25);
	23 T1.join();
	24
	25 std::cout << "After adding first value [T1.join()]: ";
	26 display();
	27
	28 std::thread T2(add, 70);
	29 T2.join();
	30
	31 std::cout << "After adding second value [T2.join()]: ";
	32 display();
	33
	34 std::thread T3(add, 1000);
	35 T3.join();
	36
	37 std::cout << "After adding the last value [T3.join()]: ";
	38 display();
	39
	40 return 0;
	41 }
	Original Value: 0
	After adding first value [T1.join()]: 25
	After adding second value [T2.join()]: 95
	After adding the last value [T3.join()]: 1095
	PS C:\DSA\HOA 13>

Analysis	The program starts with a global variable to indicate that the starting value is 0. Then with the add() function, each thread adds a different integer value: 25, 70, and 1000. The join() function makes the main thread wait for the current thread to finish before displaying the updated total. They are called one-by-one sequentially and are protected by the

	function <mutex> from the process.

Conclusion

- In this activity, the threads in C++ are created using std::thread, while join() makes the program wait for a thread to finish. I also learned that threads may run in different orders, so their output is not always predictable. Using a mutex helps protect shared variables and prevents errors when multiple threads access the same data. Lastly, I understood that concurrency and parallelism are related but different concepts in codes.
- To conclude this activity, multithreading in C++ shows how each thread can be executed differently. Just like simple-threading and multithreading. Some outputs, when called multithreading, allow multiple tasks to make progress within the same period unlike simple-threading where instructions are generally executed sequentially. Through this activity, I learned how join() waits for a thread to finish and how mutexes help protect shared variables from data races.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

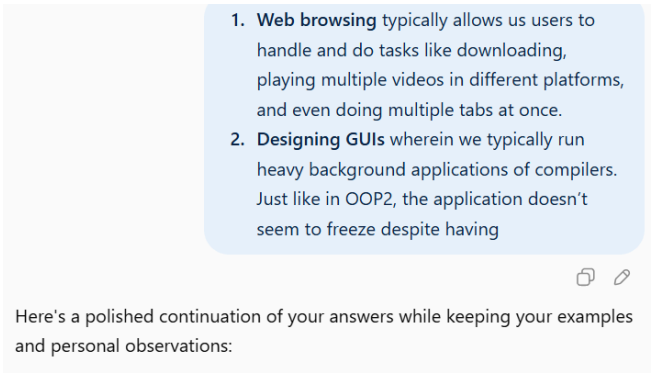
Tools Used: **Google Overview and ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **grammar correction and explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:



Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.